

INDEX.HTML

```
<!doctype html>
```

```
<html>
```

```
<head>
```

```
  <meta charset="utf-8">
```

```
  <title>SmartMed BLE - Dashboard</title>
```

```
<style>
```

```
  * { margin:0; padding:0; box-sizing:border-box; font-family:"Segoe UI",system-ui; }
```

```
  body {
```

```
    background:#eef3f8;
```

```
    padding:26px;
```

```
    max-width:1100px;
```

```
    margin:auto;
```

```
  }
```

```
  h2 {
```

```
    text-align:center;
```

```
    margin-bottom:10px;
```

```
    font-size:32px;
```

```
    font-weight:700;
```

```
    color:#214c77;
```

```
  }
```

```
  #statusBadge {
```

```
    text-align:center;
```

```
    margin-bottom:20px;
```

```
font-size:18px;
font-weight:600;
padding:8px 14px;
display:inline-block;
border-radius:8px;
background:#ccc;
color:#333;
}
```

```
.connected { background:#0fa958 !important; color:white !important; }
.disconnected { background:#cf4332 !important; color:white !important; }
.waiting { background:#6c757d !important; color:white !important; }
```

```
.card {
background:white;
padding:18px 22px;
border-radius:12px;
box-shadow:0px 2px 7px rgba(0,0,0,0.08);
margin-bottom:20px;
}
```

```
input, button {
padding:10px;
width:100%;
border-radius:8px;
border:1px solid #d0d7de;
font-size:15px;
margin-top:8px;
```

```
}
```

```
button {  
  background:#1976d2;  
  color:white;  
  border:none;  
  cursor:pointer;  
  font-weight:600;  
  transition:0.2s;  
}
```

```
button:hover:not(:disabled) { background:#135ba1; }  
button:disabled { background:#9ebedf; cursor:not-allowed; }
```

```
#connectBtn { background:#1b5e20; }  
#connectBtn:hover { background:#144619; }
```

```
ul { padding-left:0; list-style:none; }  
li {  
  margin-bottom:10px;  
  background:#f1f7ff;  
  padding:12px;  
  border-radius:10px;  
  border-left:4px solid #1976d2;  
  display:flex;  
  justify-content:space-between;  
  align-items:center;  
}
```

```
.deleteBtn {
  background:#d9534f;
  padding:6px 10px;
  width:auto;
  margin-left:10px;
  border-radius:6px;
}
.deleteBtn:hover { background:#b83532; }
```

```
#log {
  height:240px;
  overflow-y:auto;
  background:#f8fafc;
  padding:12px;
  border-radius:10px;
  border:1px solid #d5e3ef;
  white-space:pre-wrap;
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<h2>SmartMed BLE Dashboard</h2>
```

```
<!-- Connection status -->
```

```
<div id="statusBadge" class="waiting">🔄 Waiting for connection...</div>
```

```
<!-- CONNECT -->
```

```
<div class="card">
```

```
  <button id="connectBtn">🔗 Connect to ESP32 (BLE)</button>
```

```
</div>
```

```
<!-- CREATE REMINDER -->
```

```
<div class="card">
```

```
  <h3>Create New Reminder</h3>
```

```
  <input id="patient" placeholder="Patient name">
```

```
  <input id="med" placeholder="Medicine name">
```

```
  <input id="dosage" placeholder="Dosage (optional)">
```

```
  <label style="margin-top: 12px;">Schedule Date & Time</label>
```

```
  <input id="schedule" type="datetime-local">
```

```
  <div style="display:flex; gap:14px; margin-top:14px;">
```

```
    <button id="sendBtn" disabled>📅 Schedule Reminder</button>
```

```
    <button id="startNowBtn" disabled>⚡ Start Now</button>
```

```
  </div>
```

```
</div>
```

```
<!-- REMINDERS -->
```

```
<div class="card">
```

```
  <h3>🕒 Scheduled Reminders</h3>
```

```
  <button id="listBtn" disabled>🔄 Refresh List</button>
```

```
  <ul id="remList"></ul>
```

</div>

<!-- HISTORY -->

<div class="card">

<h3> 📜 History</h3>

<div style="display:flex; gap:14px;">

<button id="historyBtn" disabled> 📄 Load History</button>

<button id="clearHistoryBtn" disabled> 🗑️ Clear History</button>

</div>

<ul id="historyList">

</div>

<!-- LOG -->

<div class="card">

<h3> 📄 Log Output</h3>

<div id="log"></div>

</div>

<script src="ble_frontend.js"></script>

</body>

</html>

FRONTEND.JS

const SERVICE_UUID = "4fafc201-1fb5-459e-8fcc-c5c9c331914b";

const CMD_UUID = "beb5483e-36e1-4688-b7f5-ea07361b26a8";

const STAT_UUID = "d06b9f92-2b3d-4f6c-9b3f-2e7a1a5b4f9c";

```

let device, server, service, cmdChar, statChar;

// =====
// DOM Elements
// =====
const logEl = document.getElementById("log");
const remListEl = document.getElementById("remList");
const histListEl = document.getElementById("historyList");

// =====
// UI Status Badge
// =====
function setStatus(text, cls) {
    const el = document.getElementById("statusBadge");
    el.textContent = text;
    el.className = cls;
}

// =====
// Logging Utility
// =====
function log(msg) {
    const line = `${new Date().toLocaleTimeString()} — ${msg}\n`;
    logEl.textContent += line;
    logEl.scrollTop = logEl.scrollHeight;
    console.log(msg);
}

```

```

// =====
// Connect Button Handler
// =====

document.getElementById("connectBtn").onclick = async () => {
  try {
    setStatus("🔄 Connecting...", "waiting");
    log("Requesting BLE device...");

    device = await navigator.bluetooth.requestDevice({
      filters: [{ namePrefix: "SmartMed" }],
      optionalServices: [SERVICE_UUID]
    });

    device.addEventListener("gattserverdisconnected", onDisconnected);

    server = await device.gatt.connect();
    log("Connected to " + device.name);
    setStatus("🟢 Connected to ESP32", "connected");

    service = await server.getPrimaryService(SERVICE_UUID);
    cmdChar = await service.getCharacteristic(CMD_UUID);

    statChar = await service.getCharacteristic(STAT_UUID);

    await statChar.startNotifications();
    statChar.addEventListener("characteristicvaluechanged", onNotify);
  }
};

```



```

enableButtons(true);

await sendCmd({ cmd: "list" });

} catch (err) {
  log("Connect error: " + err);
  setStatus("● Failed to connect", "disconnected");
}
};

// =====
// Button Enable/Disable
// =====
function enableButtons(enable) {
  document.getElementById("sendBtn").disabled = !enable;
  document.getElementById("startNowBtn").disabled = !enable;
  document.getElementById("listBtn").disabled = !enable;
  document.getElementById("historyBtn").disabled = !enable;
  document.getElementById("clearHistoryBtn").disabled = !enable;
}

// =====
// BLE Disconnect Handler
// =====
function onDisconnected() {
  log("Device disconnected");
  setStatus("● Disconnected", "disconnected");
  enableButtons(false);
}

```

```

}

// =====
// Send Command to ESP32
// =====
async function sendCmd(obj) {
  if (!cmdChar) return log("Command characteristic not available");
  const data = new TextEncoder().encode(JSON.stringify(obj));
  await cmdChar.writeValue(data);
  log("CMD -> " + JSON.stringify(obj));
}

// =====
// Handle Notifications
// =====
function onNotify(event) {
  let msg = new TextDecoder().decode(event.target.value);
  log("NOTIFY: " + msg);

  try {
    const obj = JSON.parse(msg);

    if (obj.type === "list") renderList(obj.reminders);
    else if (obj.type === "history") renderHistory(obj.history);
    else if (obj.type === "scheduled") alert("Reminder scheduled!");
  } catch (err) {
    log("Notify parse error: " + err);
  }
}

```

```

}

// =====
// Schedule Reminder
// =====

document.getElementById("sendBtn").onclick = async () => {
  const timestamp = new Date(document.getElementById("schedule").value).getTime();
  if (!timestamp) return alert("Choose date & time");

  await sendCmd({
    cmd: "schedule",
    patient: document.getElementById("patient").value,
    med: document.getElementById("med").value,
    dosage: document.getElementById("dosage").value,
    timestamp,
    now: Date.now()
  });

  setTimeout(() => sendCmd({ cmd: "list" }), 600);
};

// =====
// Other Button Handlers
// =====

document.getElementById("startNowBtn").onclick = () =>
  sendCmd({ cmd: "startNow" });

document.getElementById("listBtn").onclick = () =>

```

```
sendCmd({ cmd: "list" });
```

```
document.getElementById("historyBtn").onclick = () =>
```

```
sendCmd({ cmd: "getHistory" });
```

```
document.getElementById("clearHistoryBtn").onclick = () =>
```

```
sendCmd({ cmd: "clearHistory" });
```

```
// =====
```

```
// RENDER REMINDERS (with delete button)
```

```
// =====
```

```
function renderList(reminders) {
```

```
  remListEl.innerHTML = "";
```

```
  if (!Array.isArray(reminders)) return;
```

```
  reminders.forEach(r => {
```

```
    const li = document.createElement("li");
```

```
    const dt = r.timestampMs
```

```
      ? new Date(r.timestampMs).toLocaleString()
```

```
      : "—";
```

```
    li.innerHTML = `
```

```
      <div>
```

```
        <strong>${r.med}</strong> (${r.patient})<br>
```

```
        <small>${dt}</small><br>
```

```
        <small>Status: ${r.status}</small>
```

```
      </div>
```

```
`;
```

```
// delete button
```

```
const del = document.createElement("button");
```

```
del.className = "deleteBtn";
```

```
del.textContent = "Delete";
```

```
del.onclick = () => {
```

```
  sendCmd({ cmd: "delete", id: r.id });
```

```
  setTimeout(() => sendCmd({ cmd: "list" }), 300);
```

```
};
```

```
li.appendChild(del);
```

```
remListEl.appendChild(li);
```

```
});
```

```
}
```

```
// =====
```

```
// RENDER HISTORY
```

```
// =====
```

```
function renderHistory(history) {
```

```
  histListEl.innerHTML = "";
```

```
  if (!Array.isArray(history)) return;
```

```
  history.forEach(h => {
```

```
    const dt = new Date(h.timestampMs).toLocaleString();
```

```
    const li = document.createElement("li");
```

```
    li.innerHTML = `${h.med}</strong> — ${dt} — result: ${h.result}`;
```

```
    histListEl.appendChild(li);  
  });  
}
```

```
log("Frontend ready");
```